

Blatt 4

Kryptologie

Luis Staudt

RSA knacken

Aufgabenstellung

Es soll ein mit AES-CBC verschlüsselter Ciphertext entschlüsselt werden. Der 128-Bit AES-Schlüssel wurde mit RSA verschlüsselt. Der öffentliche RSA-Schlüssel besteht aus einem 4096-Bit Modulus n (aus einem QR-Code) und $e = 65537$. Der RSA-Ciphertext steht in einem zweiten QR-Code.

- AES-Ciphertext (Base64): 0CZaBxssu0HVEGnc10auq670zKcEqQ1jtU7x+fq/L28=
- AES-Modus: CBC mit IV = 128-Bit-ALL-ZERO und 1-0-Padding

Es soll ermittelt werden, welche p - q -Konstellation die Faktorisierung des 4096-Bit Modulus ermöglicht.

Vorgehensweise

1. **QR-Codes auslesen:** Extraktion der Bilder aus dem PDF mittels `pdfimages` und Dekodierung mit OpenCV (`QRCodeDetector`).
2. **Faktorisierung:** Faktorisierung von n mittels Fermats Methode.
3. **RSA-Entschlüsselung:** Berechnung des privaten Schlüssels d und Entschlüsselung des AES-Schlüssels.
4. **AES-Entschlüsselung:** Entschlüsselung des Ciphertexts mit AES-CBC und Entfernung des 1-0-Paddings.

Die Implementierung erfolgt in Java unter Verwendung von `java.math.BigInteger` für die RSA-Arithmetik und der Bouncy-Castle-Bibliothek für AES-CBC.

Ergebnisse

Faktorisierung

Die Fermats Faktorisierungsmethode beruht darauf, $n = a^2 - b^2 = (a + b)(a - b)$ zu schreiben. Man startet mit $a = \lceil \sqrt{n} \rceil$ und prüft, ob $a^2 - n$ ein perfektes Quadrat ist.

```

1 BigInteger a = n.sqrt().add(BigInteger.ONE);
2 while (true) {
3     BigInteger bSq = a.multiply(a).subtract(n);
4     BigInteger[] sqrtResult = bSq.sqrtAndRemainder();
5     if (sqrtResult[1].equals(BigInteger.ZERO)) {
6         p = a.add(sqrtResult[0]);
7         q = a.subtract(sqrtResult[0]);
8         break;
9     }
10    a = a.add(BigInteger.ONE);
11 }

```

Code Listing 1: Fermat-Faktorisierung

Die Faktorisierung gelingt **in nur einer einzigen Iteration**:

Parameter	Wert
n (Bitlänge)	4096 Bit
p (Bitlänge)	2048 Bit
q (Bitlänge)	2048 Bit
$ p - q $ (Bitlänge)	996 Bit
Iterationen	1

Tabelle 1: Ergebnis der Faktorisierung

$p = 24569000803103 \dots 213748974553$ (617 Dezimalstellen)

$q = 24569000803103 \dots 688297261363$ (617 Dezimalstellen)

RSA-Entschlüsselung

Mit $\varphi(n) = (p - 1)(q - 1)$ und $d = e^{-1} \bmod \varphi(n)$:

$$k = c^d \bmod n$$

Parameter	Wert
d (Bitlänge)	4094 Bit
k (AES-Schlüssel, hex)	DEF0123456789ABCDEF0123456789ABC

Tabelle 2: RSA-Entschlüsselungsergebnis

AES-Entschlüsselung

Parameter	Wert
AES-Schlüssel	DEF0123456789ABCDEF0123456789ABC
IV	00000000000000000000000000000000
Modus	CBC, 1-0-Padding
Plaintext	Decryption made easy

Tabelle 3: AES-Entschlüsselungsergebnis

Schwachstellenanalyse

Warum die Faktorisierung gelingt

Die Fermats Methode benötigt $\mathcal{O}\left(\frac{|p-q|}{\sqrt{n}}\right)$ Iterationen. In diesem Fall:

- $|p - q|$ hat nur 996 Bit, obwohl beide Primzahlen jeweils 2048 Bit lang sind.
- $\sqrt{n}$ hat 2048 Bit.
- $\frac{|p-q|}{2\sqrt{n}} \approx 2^{996}/2^{2049} = 2^{-1053} \ll 1$, daher genügt **eine einzige Iteration**.

Die Ursache ist, dass p und q extrem nahe beieinander liegen. Ihre oberen ~ 1052 Bit sind identisch – sie unterscheiden sich erst in den unteren 996 Bit. Damit ist $a = \lceil \sqrt{n} \rceil \approx \frac{p+q}{2}$ nahezu exakt, und $b^2 = a^2 - n = \left(\frac{p-q}{2}\right)^2$ ist sofort ein perfektes Quadrat.

Grenzen der Konstellation

Fermats Methode ist genau dann effizient, wenn $|p - q|$ klein relativ zu $n^{1/4}$ ist:

- Ist $|p - q| < n^{1/4} \approx 2^{1024}$, so gelingt die Faktorisierung in wenigen Iterationen.
- Hier: $|p - q| \approx 2^{996} < 2^{1024} = n^{1/4}$ – deutlich innerhalb der Grenze.

- Selbst bei $|p - q| \approx 2^{1024}$ wäre Fermats Methode noch praktikabel.
- Bei $|p - q| > 2^{1024}$ wächst die Iterationszahl exponentiell und die Methode wird unpraktikabel.

Gegenmaßnahmen

Um diese Schwachstelle zu vermeiden:

- p und q müssen unabhängig und gleichverteilt aus dem Bereich $(2^{n/2-1}, 2^{n/2})$ gewählt werden.
- Es muss sichergestellt werden, dass $|p - q| > 2^{n/4}$ (bei 4096-Bit RSA: $|p - q| > 2^{1024}$).
- Moderne Schlüsselgenerierungen (z. B. nach FIPS 186-4) prüfen explizit, dass $|p - q| > 2^{n/2-100}$, um Fermats Angriff auszuschließen.

Bewertung

Die Aufgabe demonstriert eindrücklich, dass die Sicherheit von RSA nicht allein von der Schlüssellänge abhängt. Obwohl ein 4096-Bit Modulus verwendet wird, genügt eine einzige Iteration der Fermat-Faktorisierung, um n zu zerlegen.

Die Schwachstelle liegt in der Wahl von p und q : Diese Primzahlen liegen so nahe beieinander, dass $\lceil \sqrt{n} \rceil$ praktisch mit $\frac{p+q}{2}$ übereinstimmt. Dies zeigt, dass die zufällige und unabhängige Wahl der Primfaktoren eine notwendige Voraussetzung für die RSA-Sicherheit ist – Größe allein reicht nicht aus.